

Fundamentos de la programación

Laboratorio. Hoja 5

Los ejercicios marcados como **Evaluable** tienen que subirse al CV durante la sesión de laboratorio.

- Se subirá **únicamente** el archivo **Program.cs**
- Sólo hará la entrega uno de los miembros del grupo.
- La línea 1 (y siguientes) deben contener los nombres de los alumnos que hacen la entrega de la forma:

```
// Nombre Apellido1 Apellido2
// Nombre Apellido1 Apellido2
```

-
1. **(Evaluable)** Vamos a implementar la clase **SetCoor** de *conjuntos de coordenadas*, utilizando la clase **Coor** vista en clase. Para representar conjuntos, en esta implementación, utilizaremos un array **coors** que almacenará coordenadas agrupadas en las primeras posiciones del mismo (análogo a la representación de los polinomios) y un entero **oc** que indicará el número de componentes ocupadas (y que coincide con la primera posición libre en el array). La estructura de la clase tendrá el siguiente aspecto:

```
using Coordinates;
namespace SetArray;

class SetCoor{
    // atributos de la clase
    Coor [] coors; // array con coordenadas
    int oc;        // núm de componentes ocupadas del array = primera pos libre
}
```

Los métodos a implementar son los siguientes (todos públicos salvo que se indique expresamente lo contrario en el prototipo):

- **SetCoor(int tam=10)**: crea el array **coors** con el tamaño dado (10 por defecto) e inicializa **oc**.
- **private int SearchElem(Coor c)**: busca una coordenada dada en el array **coors**. Si está devuelve su posición en dicho array, -1 en otro caso. Este método será útil para los dos siguientes.
- **bool Add(Coor c)**: añade una coordenada dada al conjunto. Si esa coordenada ya está, no modifica el conjunto y devuelve **false**. Si no está y hay espacio en el array, la añade y devuelve **true**; si no cabe, lanza un mensaje de error y no modifica el conjunto.
- **bool Belongs(Coor c)**: comprueba si la coordenada **c** pertenece al conjunto (trivial utilizando **SearchElem**).

Para comprobar el funcionamiento el método **Main** debe implementar un pequeño repertorio de pruebas con los distintos métodos.

2. Extender la clase anterior para incluir los siguientes métodos:

- **ToString**: conversión a texto para escritura en pantalla (útil para depuración).
- **Remove**: elimina una coordenada dada del conjunto y devuelve **true**, si esa coordenada está en el conjunto; no modifica el conjunto y devuelve **false** en caso contrario.
- **PopElem**: extrae (elimina) un elemento del conjunto y lo devuelve. Como no hay orden, debe extraerse el que suponga menor coste computacional. Si el conjunto es vacío debe lanzar una excepción.
- **Size**: devuelve el número de elementos del conjunto.
- Incluir los operadores **+**, **-**, **&** para que realicen la unión y la diferencia de conjuntos, respectivamente.
- Incluir los operadores **==** y **!=** para la igualdad y desigualdad de conjuntos.
- **bool IsSubset(SetCoor s)**: devuelve **true** si el conjunto actual es subconjunto de **s**, **false** en otro caso.
- **SetCoor Copy()**: devuelve una copia del conjunto actual.